

STM32 I2C – Sequence of Actions

Ankit Srivastava

EE2028

STM32L4S5xx Datasheet

STM32L4+ User Manual RM0432

B-L4S5I-IOT01A User Manual UM2708

HTS221 Datasheet

Acknowledgement: Slides adapted from Dr Rajesh Panicker



OVERVIEW

- Sequence of Actions
 - Interplay between CR2 and ISR registers
 - Reading and Writing slave registers
 - Autoend and Reload modes
- Code Walkthrough

STM32 I2C: I2C_CR2 AND I2C_ISR

Address offset: 0x04,
Default value: 0x0000 0000

read or set. Don't clear
(self-clearing)

I2C_CR2: Control Register 2

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	PEC BYTE	AUTOE ND	RE LOAD	NBYTES[7:0]							
					rs	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NACK	STOP	START	HEAD1 OR	ADD10	RD_ WRN	SADD[9:0]									
rs	rs	rs	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Address offset: 0x18,
Default value: 0x0000 0001

I2C_ISR: Interrupt and Status Register

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ADDCODE[6:0]						DIR	
								r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BUSY	Res.	ALERT	TIME OUT	PEC ERR	OVR	ARLO	BERR	TCR	TC	STOPF	NACKF	ADDR	RXNE	TXIS	TXE
r		r	r	r	r	r	r	r	r	r	r	r	r	rs	rs

STM32 I2C: SEQUENCE OF ACTIONS

Writing 1 byte (AUTOEND)

$\overline{\text{BUSY}}$ | S > DevAddress_W > $\overline{\text{START=0}}$ | A > WriteData > $\overline{\text{TXIS}}$ | A > P | $\overline{\text{STOPF}}$
START=1, SADD=DevAddress, RD_WRN=0, NBYTES=1, AUTOEND=1, RELOAD=0 | TXDR=WriteData | STOPCF=1, Reset CR2

Reading 1 byte (AUTOEND)

$\overline{\text{BUSY}}$ | S > DevAddress_R > $\overline{\text{START=0}}$ | A > ReadData > $\overline{\text{RXNE}}$ | A' > P | $\overline{\text{STOPF}}$
START=1, SADD=DevAddress, RD_WRN=1, NBYTES=1, AUTOEND=1, RELOAD=0 | ReadData=RXDR | STOPCF=1, Reset CR2

Writing 1 slave register (AUTOEND)

$\overline{\text{BUSY}}$ | S > DevAddress_W > $\overline{\text{START=0}}$ | A > RegAddress > $\overline{\text{TXIS}}$ | A > RegData > $\overline{\text{TXIS}}$ | A > P | $\overline{\text{STOPF}}$
START=1, SADD=DevAddress, RD_WRN=0, NBYTES=2, AUTOEND=1, RELOAD=0 | TXDR=RegAddress | TXDR=RegData | STOPCF=1, Reset CR2

Status flag set by h/w, polled by s/w in red.
Action by s/w in response to flag in green.
SCL is stretched low by h/w while waiting for s/w action.
Auto-clearing action of control bits by h/w in orange.

Master to Slave in black
Slave to Master in blue

Dev address = Device address =
Slave address = Target address

Software = User Program/Code
Hardware = STM32 I2C hardware

I2C Master => STM32 I2C peripheral =
STM32 I2C controller = STM32 I2C hardware
*Note that STM32 I2C can be configured as slave but we are not using STM32 I2C slave mode in any of our examples

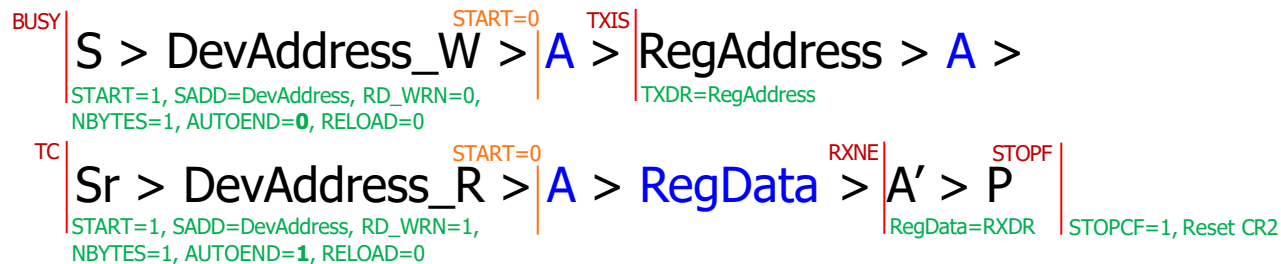
STM32 I2C: SEQUENCE OF ACTIONS

Writing 1 slave register (AUTOEND=0, i.e., software end mode)



Using AUTOEND involves less software overhead; software end mode generally used only when there is a potential of change of direction after the current read/write

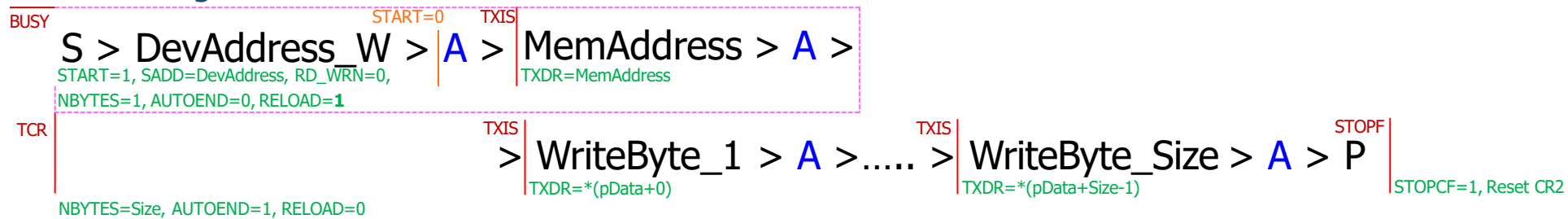
Reading 1 slave register



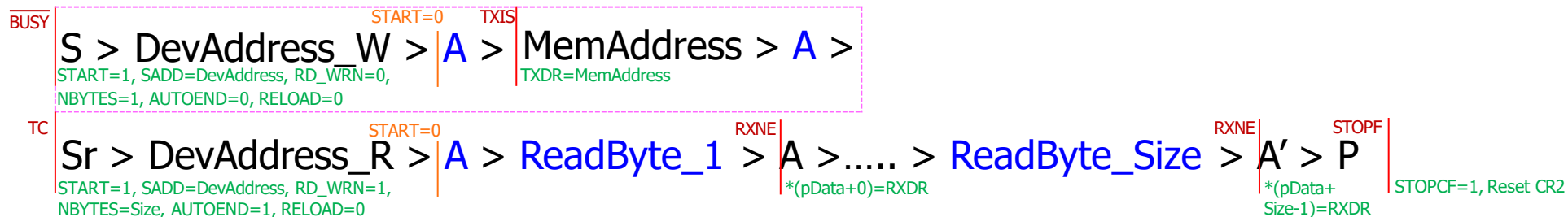
STM32 I2C: SEQUENCE OF ACTIONS

Writing Size (≤ 255) bytes via autoincrement (Size excl. MemAddress byte)

- RELOAD mode is used for the write below, but it could be done without RELOAD if Size ≤ 254 , as 1 byte is needed for MemAddress byte (i.e., the slave register address)
- Using RELOAD allows the driver functions for read and write to be similar



Reading Size (≤ 255) bytes via autoincrement



SENSOR IO READ

```
/*stm32l4s5i_iot01.c*/
uint8_t SENSOR_IO_Read(uint8_t Addr, uint8_t Reg)
{
    uint8_t read_value = 0;

    I2Cx_ReadMultiple(&hI2cHandler, Addr, Reg, I2C_MEMADD_SIZE_8BIT, (uint8_t*)&read_value, 1);

    return read_value;
}

uint16_t SENSOR_IO_ReadMultiple(uint8_t Addr, uint8_t Reg, uint8_t *Buffer, uint16_t Length)
{
    return I2Cx_ReadMultiple(&hI2cHandler, Addr, (uint16_t)Reg, I2C_MEMADD_SIZE_8BIT, Buffer, Length);
}

static HAL_StatusTypeDef I2Cx_ReadMultiple(I2C_HandleTypeDef *i2c_handler, uint8_t Addr,
                                           uint16_t Reg, uint16_t MemAddress, uint8_t *Buffer, uint16_t Length)
{
    HAL_StatusTypeDef status = HAL_OK;

    status = HAL_I2C_Mem_Read(i2c_handler, Addr, (uint16_t)Reg, MemAddress, Buffer, Length, 1000);

    /* Check the communication status */
    if(status != HAL_OK)
    {
        /* I2C error occurred */
        I2Cx_Error(i2c_handler, Addr);
    }
    return status;
}
```

BUSY

S > DevAddress_W > A > MemAddress > A >

START=1, SADD=DevAddress, RD_WRN=0,
NBYTES=1, AUTOEND=0, RELOAD=0

TXIS
TXDR=MemAddress

TC

Sr > DevAddress_R > A > ReadByte_1 > A > > ReadByte_Size > A' > P

START=0
START=1, SADD=DevAddress, RD_WRN=1,
NBYTES=Size, AUTOEND=1, RELOAD=0

RXNE

*(pData+0)=RXDR

RXNE

*(pData+
Size-1)=RXDR

STOPF

STOPCF=1, Reset CR2

```
HAL_StatusTypeDef HAL_I2C_Mem_Read(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t
    MemAddSize, uint8_t *pData, uint16_t Size, uint32_t Timeout)
{
    // Size is the num. bytes to be read
    // MemAddSize is the num. bytes needed to represent a slave reg. addr. This is assumed to be 1 (as with most slaves)
    uint32_t tickstart = HAL_GetTick();
    if (I2C_WaitOnFlagUntilTimeout(hi2c, I2C_FLAG_BUSY, SET, I2C_TIMEOUT_BUSY, tickstart) != HAL_OK) {return HAL_TIMEOUT;}
    // Send S, Slave Address, Address of register within the slave to be read
    I2C_RequestMemoryRead(hi2c, DevAddress, MemAddress, MemAddSize, Timeout, tickstart);
    // Send Sr, Slave Address, load NBYTES assuming hi2c->XferCount < 256. Writes to I2C_CR2.
    I2C_TransferConfig(hi2c, DevAddress, Size, I2C_AUTOEND_MODE, I2C_GENERATE_START_READ);

    do // Receive 'Size' bytes
    {
        // for the sensors on the board, hi2c->Instance = I2C2
        // Wait until RXNE flag in I2C_ISR is set
        if (I2C_WaitOnFlagUntilTimeout(hi2c, I2C_FLAG_RXNE, RESET, Timeout, tickstart) != HAL_OK) {return HAL_TIMEOUT;}
        (*pData++) = hi2c->Instance->RXDR; // Read data from I2C_RXDR
        Size--;
        // Need to wait until TCR flag in I2C_ISR is set and reload NBYTES if Size > 255 (code not shown)
    } while (Size > 0U);

    // No need to Check TC flag in I2C_ISR, with AUTOEND mode the stop is automatically generated
    // Wait until STOPF flag in I2C_ISR is set
    if (I2C_WaitOnSTOPFlagUntilTimeout(hi2c, Timeout, tickstart) != HAL_OK) {return HAL_TIMEOUT;}

    __HAL_I2C_CLEAR_FLAG(hi2c, I2C_FLAG_STOPF); // Clear STOP Flag
    I2C_RESET_CR2(hi2c); // Clear I2C_CR2
    return HAL_OK;
}
```

*Note that the code shown here is taken from the HAL library of another STM32 chip — specifically, the STM32L475. While there might be some minor differences when compared to the HAL library for the STM32L4S5 chip that we use in the lab, the core logic and initialization flow remain the same.

Ideally, the HAL library should be portable across different STM32 chips. However, small variations can exist between versions. These are often introduced due to performance optimizations, software bug fixes, or feedback from developers in subsequent library releases

BUSY

S > DevAddress_W > A > MemAddress > A >

START=1, SADD=DevAddress, RD_WRN=0,
NBYTES=1, AUTOEND=0, RELOAD=0

TXIS
TXDR=MemAddress

TC

Sr > DevAddress_R > A > ReadByte_1 > A > > ReadByte_Size > A' > P

START=1, SADD=DevAddress, RD_WRN=1,
NBYTES=Size, AUTOEND=1, RELOAD=0

RXNE

*(pData+0)=RXDR

RXNE

*(pData+
Size-1)=RXDR

STOPF

STOPCF=1, Reset CR2

```
HAL_StatusTypeDef HAL_I2C_Mem_Read(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t
*pData, uint16_t Size, uint32_t Timeout)
{
    // Size is the num. bytes to be read
    // MemAddSize is the num. bytes needed to represent a slave reg. addr. This is assumed to be 1 (as with most slaves)
    // for the sensors on the board, hi2c->Instance = I2C2

    uint32_t tmp;

    while(hi2c->Instance->SR & I2C_FLAG_BUSY);    // Wait for I2C interface to be non-busy

    // Send S, Slave Address, Address of register within the slave to be read
    // ----- I2C_RequestMemoryRead() begins -----
    // -- I2C_TransferConfig() begins --
    tmp = hi2c->Instance->CR2;

    tmp &= ~(I2C_CR2_SADD | I2C_CR2_NBYTES | I2C_CR2_RELOAD | I2C_CR2_AUTOEND | (I2C_CR2_RD_WRN &
(uint32_t)(I2C_GENERATE_START_READ >> (31U - I2C_CR2_RD_WRN_Pos)))) | I2C_CR2_START | I2C_CR2_STOP);

    tmp |= (uint32_t)((((uint32_t)DevAddress & I2C_CR2_SADD) | (((uint32_t)MemAddSize << I2C_CR2_NBYTES_Pos) & I2C_CR2_NBYTES) |
(uint32_t)I2C_SOFTEND_MODE | (uint32_t)I2C_GENERATE_START_WRITE);

    hi2c->Instance->CR2 = tmp;
    // -- I2C_TransferConfig() ends --

    // Wait until TXIS in I2C_ISR flag is set
    while(!(hi2c->Instance->SR & I2C_FLAG_TXIS)); // I2C_WaitOnTXISFlagUntilTimeout()

    // Address of register within the slave to be read. Only LSB for 8-bit reg. address
    hi2c->Instance->TXDR = (uint8_t)(MemAddress & 0x00FFU);
```

BUSY

S > DevAddress_W > A > MemAddress > A >

START=1, SADD=DevAddress, RD_WRN=0,
NBYTES=1, AUTOEND=0, RELOAD=0

TXDR=MemAddress

TC

Sr > DevAddress_R > A > ReadByte_1 > A > > ReadByte_Size > A' > P

START=1, SADD=DevAddress, RD_WRN=1,
NBYTES=Size, AUTOEND=1, RELOAD=0

*(pData+0)=RXDR

*(pData+
Size-1)=RXDR

STOPCF=1, Reset CR2

```
// Wait until TC flag in I2C_ISR is set
while(!(hi2c->Instance->SR & I2C_FLAG_TC)); // I2C_WaitOnFlagUntilTimeout(.., I2C_FLAG_TC,...)
// ----- I2C_RequestMemoryRead() ends -----

// Send Sr, Slave Address, assuming hi2c->XferCount < 256. Writes to I2C_CR2.
// -- I2C_TransferConfig() begins --
tmp = hi2c->Instance->CR2;

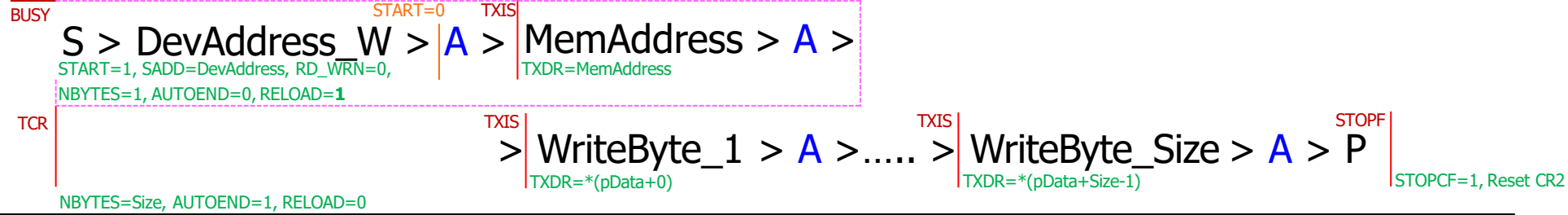
tmp &= ~(I2C_CR2_SADD | I2C_CR2_NBYTES | I2C_CR2_RELOAD | I2C_CR2_AUTOEND | (I2C_CR2_RD_WRN &
(uint32_t)(I2C_GENERATE_START_READ >> (31U - I2C_CR2_RD_WRN_Pos))) | I2C_CR2_START | I2C_CR2_STOP);

tmp |= (uint32_t)((((uint32_t)DevAddress & I2C_CR2_SADD) | (((uint32_t)Size << I2C_CR2_NBYTES_Pos) & I2C_CR2_NBYTES) |
(uint32_t)I2C_AUTOEND_MODE | (uint32_t)I2C_GENERATE_START_READ);

hi2c->Instance->CR2 = tmp;
// -- I2C_TransferConfig() ends --

do // Receive 'Size' bytes
{
    // Wait until RXNE flag in I2C_ISR is set
    while(!(hi2c->Instance->SR & I2C_FLAG_RXNE)); // I2C_WaitOnFlagUntilTimeout(.., I2C_FLAG_RXNE,...)
    (*pData++) = hi2c->Instance->RXDR; // Read data from I2C_RXDR
    Size--;
    // Need to wait until hi2c->Instance->SR & I2C_FLAG_TCR is true and reload NBYTES if Size > 255 (code not shown)
} while (Size > 0U);

// No need to Check TC flag in I2C_ISR, with AUTOEND mode the stop is automatically generated
// Wait until STOPF flag in I2C_ISR is set
while(!(hi2c->Instance->SR & I2C_FLAG_STOPF)); // I2C_WaitOnSTOPFlagUntilTimeout()
hi2c->Instance->ICR = I2C_FLAG_STOPF; // Clear STOP Flag
// Clear I2C_CR2
hi2c->Instance->CR2 &= (uint32_t)~((uint32_t)(I2C_CR2_SADD | I2C_CR2_HEAD10R | I2C_CR2_NBYTES | I2C_CR2_RELOAD | I2C_CR2_RD_WRN));
return HAL_OK;
}
```

```

HAL_StatusTypeDef HAL_I2C_Mem_Write(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t
                                   MemAddSize, uint8_t *pData, uint16_t Size, uint32_t Timeout)
{
    // Size is the num. bytes to be written, excluding slave addr and slave reg addr
    // MemAddSize is the num. bytes needed to represent a slave reg. addr. This is assumed to be 1 (as with most slaves)
    uint32_t tickstart = HAL_GetTick();
    if (I2C_WaitOnFlagUntilTimeout(hi2c, I2C_FLAG_BUSY, SET, I2C_TIMEOUT_BUSY, tickstart) != HAL_OK) {return HAL_TIMEOUT;}
    // Send S, Slave Address, Address of register within the slave to be written
    I2C_RequestMemoryWrite(hi2c, DevAddress, MemAddress, MemAddSize, Timeout, tickstart)
    // Do not send Sr, just reload NBYTES assuming hi2c->XferCount < 256. Writes to I2C_CR2
    I2C_TransferConfig(hi2c, DevAddress, Size, I2C_AUTOEND_MODE, I2C_NO_STARTSTOP);

    do // Send 'Size' bytes
    {
        // for the sensors on the board, hi2c->Instance = I2C2
        // Wait until TXIS flag in I2C_ISR is set
        if (I2C_WaitOnTXISFlagUntilTimeout(hi2c, Timeout, tickstart) != HAL_OK) {return HAL_TIMEOUT;}
        hi2c->Instance->TXDR = (*pData++); // Write data to I2C_TXDR
        Size--;
        // Need to wait until TCR flag in I2C_ISR is set and reload NBYTES if Size > 255 (code not shown)
    } while (Size > 0U);

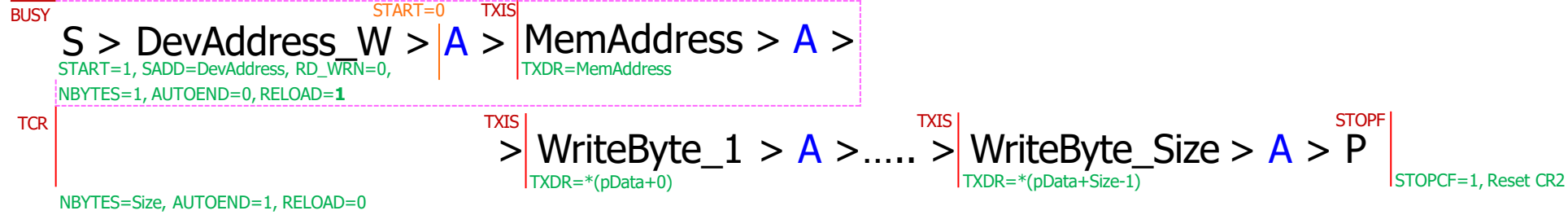
    // No need to Check TC flag in I2C_ISR, with AUTOEND mode the stop is automatically generated
    // Wait until STOPF flag in I2C_ISR is set
    if (I2C_WaitOnSTOPFlagUntilTimeout(hi2c, Timeout, tickstart) != HAL_OK) {return HAL_TIMEOUT;}

    __HAL_I2C_CLEAR_FLAG(hi2c, I2C_FLAG_STOPF); // Clear STOP Flag
    I2C_RESET_CR2(hi2c); // Clear I2C_CR2
    return HAL_OK;
}

```

*Note that the code shown here is taken from the HAL library of another STM32 chip — specifically, the STM32L475. While there might be some minor differences when compared to the HAL library for the STM32L4S5 chip that we use in the lab, the core logic and initialization flow remain the same.

Ideally, the HAL library should be portable across different STM32 chips. However, small variations can exist between versions. These are often introduced due to performance optimizations, software bug fixes, or feedback from developers in subsequent library releases



```

HAL_StatusTypeDef HAL_I2C_Mem_Write(I2C_HandleTypeDef *hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t
*pData, uint16_t Size, uint32_t Timeout)
{
    // Size is the num. bytes to be written, excluding slave addr and slave reg addr
    // MemAddSize is the num. bytes needed to represent a slave reg. addr. This is assumed to be 1 (as with most slaves)
    // for the sensors on the board, hi2c->Instance = I2C2

    uint32_t tmp;

    while(hi2c->Instance->SR & I2C_FLAG_BUSY); // Wait for I2C interface to be non-busy

    // Send S, Slave Address, Address of register within the slave to be read
    // ----- I2C_RequestMemoryRead() begins -----
    // -- I2C_TransferConfig() begins --
    tmp = hi2c->Instance->CR2;

    tmp &= ~(I2C_CR2_SADD | I2C_CR2_NBYTES | I2C_CR2_RELOAD | I2C_CR2_AUTOEND | (I2C_CR2_RD_WRN &
(uint32_t)(I2C_GENERATE_START_READ >> (31U - I2C_CR2_RD_WRN_Pos)))) | I2C_CR2_START | I2C_CR2_STOP);

    tmp |= (uint32_t)((((uint32_t)DevAddress & I2C_CR2_SADD) | (((uint32_t)MemAddSize << I2C_CR2_NBYTES_Pos) & I2C_CR2_NBYTES) |
(uint32_t)I2C_RELOAD_MODE | (uint32_t)I2C_GENERATE_START_WRITE));

    hi2c->Instance->CR2 = tmp;
    // -- I2C_TransferConfig() ends --

    // Wait until TXIS in I2C_ISR flag is set
    while(!(hi2c->Instance->SR & I2C_FLAG_TXIS)); // I2C_WaitOnTXISFlagUntilTimeout()

    // Address of register within the slave to be read. Only LSB for 8-bit reg. address
    hi2c->Instance->TXDR = (uint8_t)(MemAddress & 0x00FFU);
}

```

BUSY

S > DevAddress_W > A > MemAddress > A >

START=1, SADD=DevAddress, RD_WRN=0,

NBYTES=1, AUTOEND=0, RELOAD=1

TXDR=MemAddress

TCR

TXIS

TXIS

STOPF

> WriteByte_1 > A > > WriteByte_Size > A > P

NBYTES=Size, AUTOEND=1, RELOAD=0

TXDR=*(pData+0)

TXDR=*(pData+Size-1)

STOPCF=1, Reset CR2

```

// Wait until TCR flag in I2C_ISR is set
while(!(hi2c->Instance->ISR & I2C_FLAG_TCR)); // I2C_WaitOnFlagUntilTimeout(.., I2C_FLAG_TCR,...)
// ----- I2C_RequestMemoryRead() ends -----

// Do not send Sr, just reload NBYTES assuming hi2c->XferCount < 256. Writes to I2C_CR2
// -- I2C_TransferConfig() begins --
tmp = hi2c->Instance->CR2;

tmp &= ~(I2C_CR2_SADD | I2C_CR2_NBYTES | I2C_CR2_RELOAD | I2C_CR2_AUTOEND | (I2C_CR2_RD_WRN &
(uint32_t)(I2C_NO_STARTSTOP >> (31U - I2C_CR2_RD_WRN_Pos))) | I2C_CR2_START | I2C_CR2_STOP);

tmp |= (uint32_t)((((uint32_t)DevAddress & I2C_CR2_SADD) | (((uint32_t)Size << I2C_CR2_NBYTES_Pos) & I2C_CR2_NBYTES) |
(uint32_t)I2C_AUTOEND_MODE | (uint32_t)I2C_GENERATE_START_READ);

hi2c->Instance->CR2 = tmp;
// -- I2C_TransferConfig() ends --

do // Send 'Size' bytes
{
    // Wait until TXIS flag in I2C_ISR is set
    while(!(hi2c->Instance->ISR & I2C_FLAG_TXIS)); // I2C_WaitOnFlagUntilTimeout(.., I2C_FLAG_TXIS,...)
    hi2c->Instance->TXDR = (*pData++); // Write data to I2C_TXDR
    Size--;
    // Need to wait until hi2c->Instance->ISR & I2C_FLAG_TCR is true and reload NBYTES if Size > 255 (code not shown)
} while (Size > 0U);

// No need to Check TC flag in I2C_ISR, with AUTOEND mode the stop is automatically generated
// Wait until STOPF flag in I2C_ISR is set
while(!(hi2c->Instance->ISR & I2C_FLAG_STOPF)); // I2C_WaitOnSTOPFlagUntilTimeout()
hi2c->Instance->ICR = I2C_FLAG_STOPF; // Clear STOP Flag
// Clear I2C_CR2
hi2c->Instance->CR2 &= (uint32_t)~((uint32_t)(I2C_CR2_SADD | I2C_CR2_HEAD10R | I2C_CR2_NBYTES | I2C_CR2_RELOAD | I2C_CR2_RD_WRN));
return HAL_OK;
}

```


CONCLUSION

- Sequence of Actions
 - Interplay between CR2 and ISR registers
 - Reading and Writing slave registers
 - Autoend
 - Not enabled for Change of direction (Read after Write)
 - Enabled for all other cases
 - Reload
 - Memory (or Slave Register) Address and Bytes to be transferred
 - Bytes to be transferred > 255 bytes
- Code Walkthrough